COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

DEPARTMENT OF COMPUTER ENGINEERING

Mobile Application Systems and Design

POSTS MANAGER WITH SQLITE

Registration numbers:

- 223006944

Year 3

Lecturer: UKULIKIYEYESU (Mr Dieudonne)

Done at 24rd Mar 2026

Reg N: 22300 6944
Mobile Applications systems & Design
Date 24/03/2026

(Q1) Which dependencies did you use & Why?

- Sqflite 2.3.3 – official flutter plugin that wraps the Native Sqlite C Library on android/ios. provides Dart APIs to perform Sql operations on a local .db file.
- path 1.9.0 Constructs the correct filesystem path to the database file using getDatabasesPath() + join(), ensuring the .db file is saved in the app specific directory on each platform.

Sqlite was Necessary because of the following features:

- Offline Access: Data lives on the device
- Persistence: It survives app restarts, crashes, reboots.
- Structured Storage: uses types columns & Sql queries.
- ACID Compliance: guarantees Atomicity, Consistency, Isolation Durability.
- Lightweight & Embedded: No server needed.

(Q2) How do you handle Database Exceptions

a) Database Not Initialized

A singleton pattern with Lazy initialization ensures the database is only opened once:

```
Static Database? _Database;
Future<Database> get database async {
if (_database != null) return _database!;
_database = await _initDatabase();
return _database!; }
```

1) openDatabase() fails (permission denied, disk full), the exception propagates to the CRUD method which re-throws it as a descriptive message shown in a SnackBar

b) Insert/update/Delete errors
every write method wraps its operation in try-catch
ConflictAlgorithm.replace handles duplicate-key conflicts.
All other errors are caught and rethrown with a clear message for the UI to display.

c) Invalid or corrupted data
- NullID Guard: updatePost() and DeletePost() check post.id == null and throw immediately.
- Type-safe fromMap(): explicit bool casts throw typeerror
- Not Null Constraints
- UI validation

3) a) aDatabase is the entire sqlite file stored on the device (posts.db). It is the top level container created once with openDatabase() and holds all tables. A table is a grid of rows & columns inside the database - each row is one record, each column is a typed field. this app has one database with one table called posts. with columns Id (PK), title, body, Author, and CreatedAt)

b) CRUD operations
- Create: db.insert() - add a new row
- Read: db.query() - fetches all rows
- Update: db.update() - overwrites an existing row
- Delete: db.delete() - remove the row matched by id

c) Sqlite reads & write the device file system, which is slower than CPU operations. Doing this on the main UI thread would freeze the app. flutter solves this with Dart's async/await.

App Screenshots

All screenshots show the complete functionality of the app.

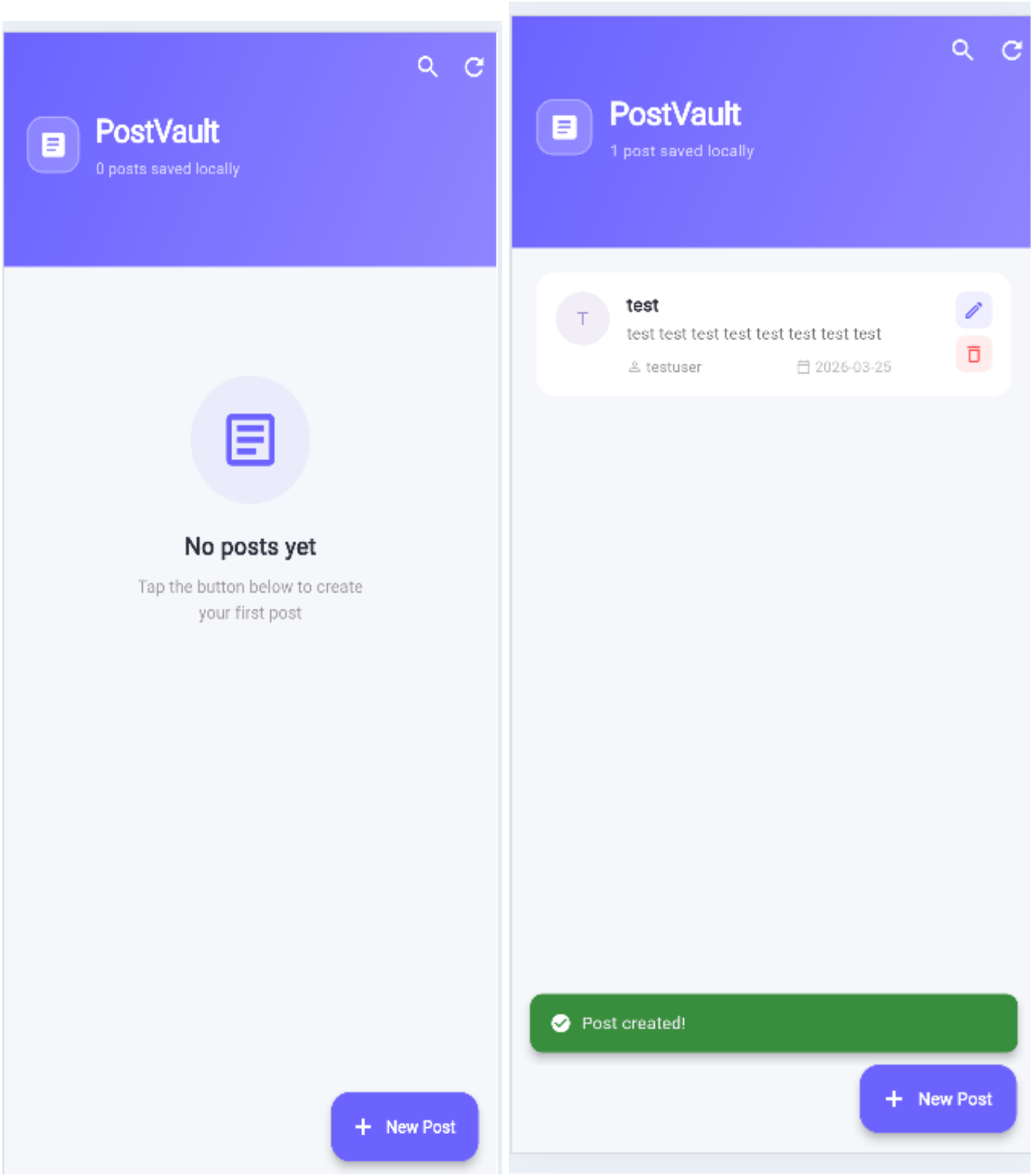

Fig1: Creating a new post

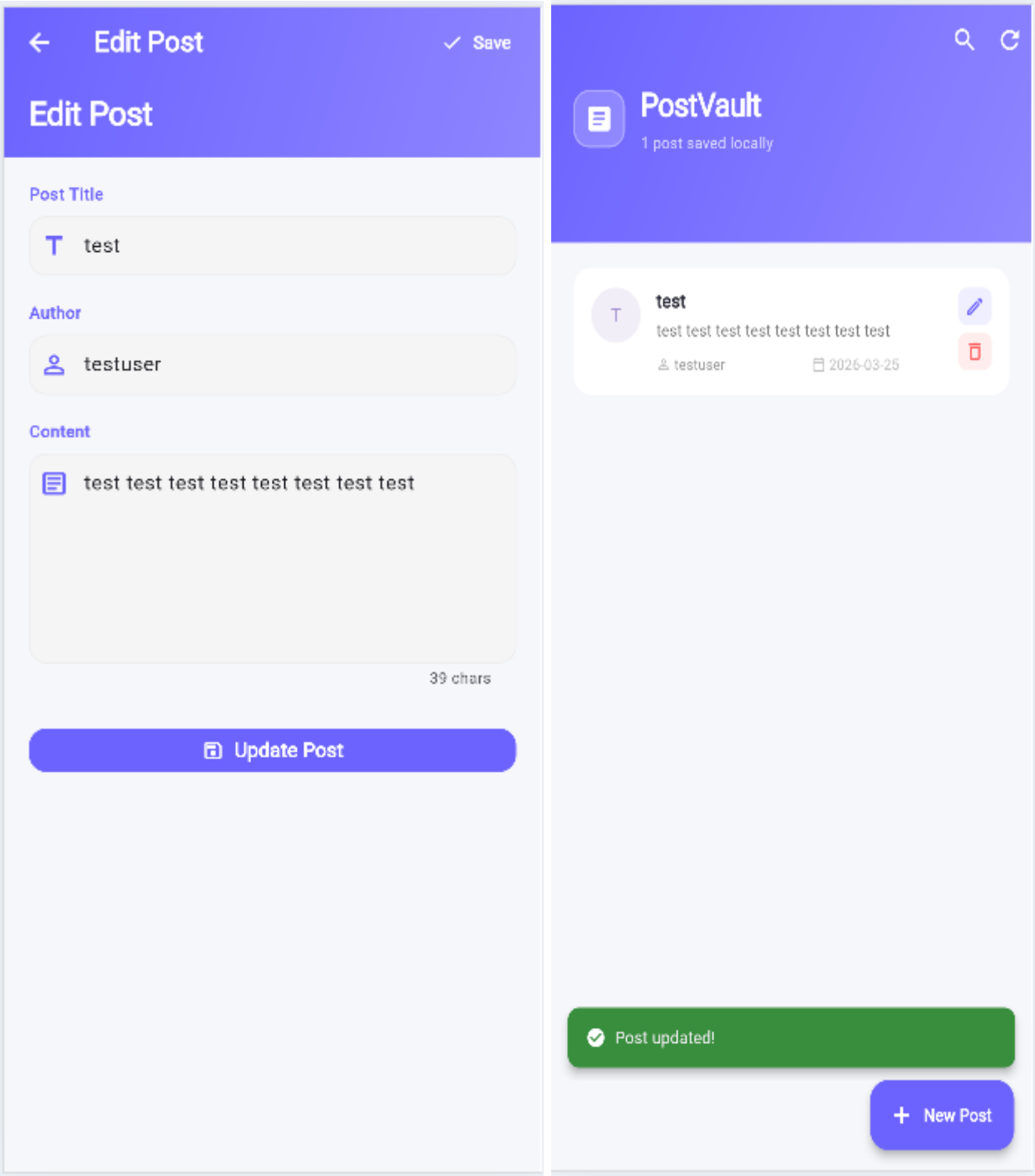

Fig 2: updating a post

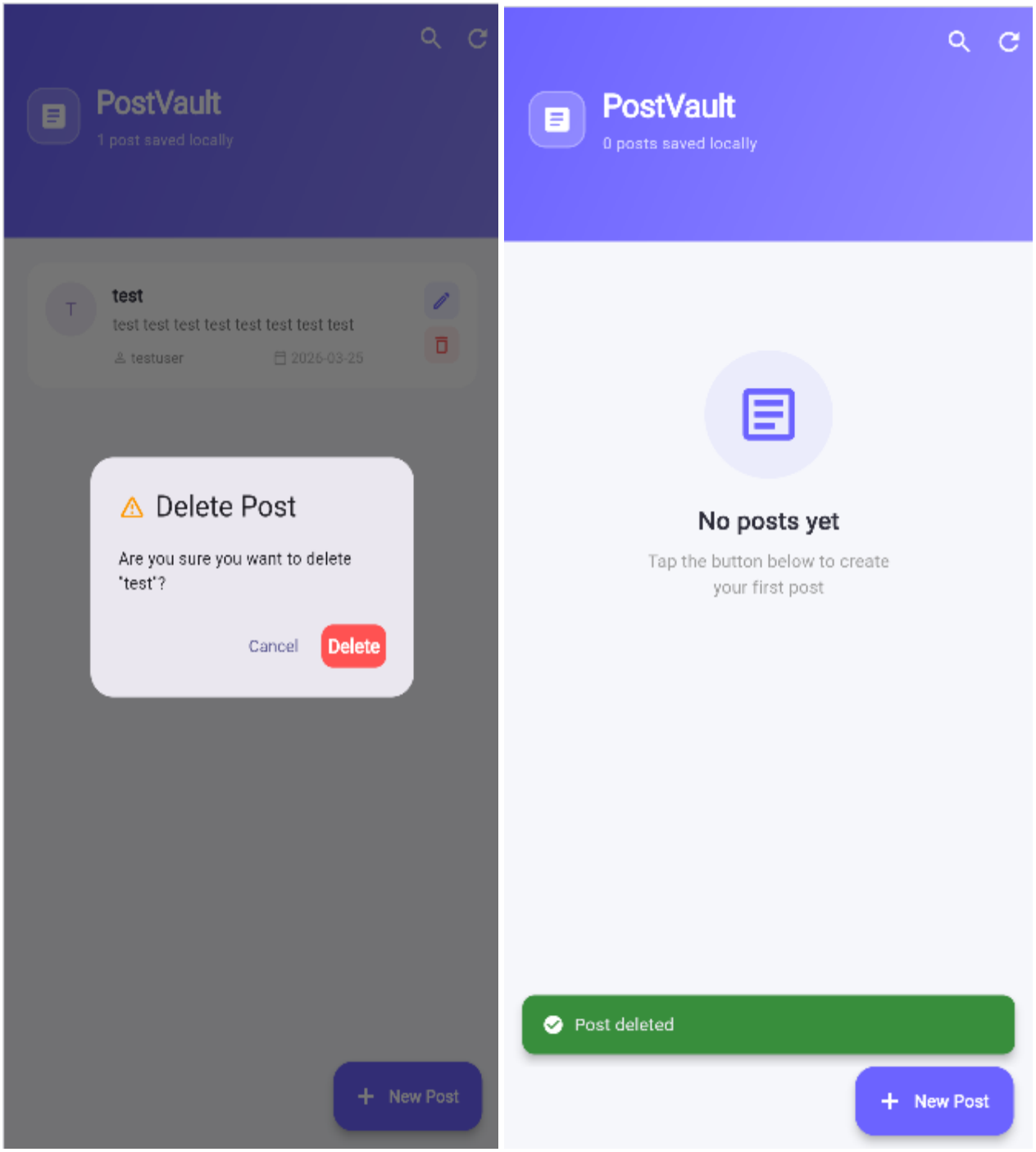

Fig 3: deleting a post

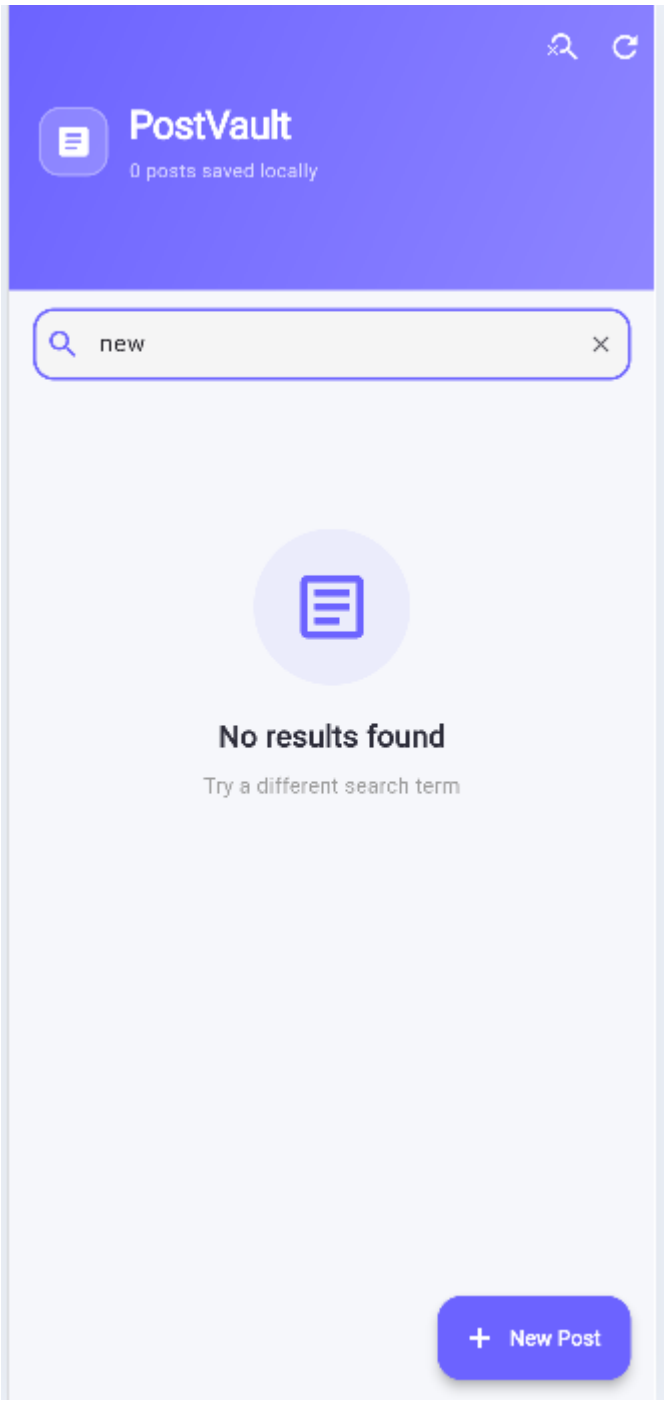

Fig 4 deleting a post

Conclusion

This lab demonstrated how Flutter applications can store and manage data locally using SQLite through the sqflite package. By building the Offline Posts Manager, I was able to implement all five CRUD operations — Create, Read, Update, and Delete — within a clean, structured codebase using the Singleton pattern for database management. The use of asynchronous programming with async/await ensured that all database operations ran on background threads, keeping the UI smooth and responsive at all times. Error handling was implemented at every layer, from database initialisation to individual operations, ensuring the app behaves predictably even when unexpected data or system issues occur. Overall, local storage with SQLite is a reliable and efficient solution for mobile apps that require offline functionality, data persistence, and structured querying without depending on a remote server.

References

1. Flutter Team. (2024). *sqflite package — Flutter & Dart*. pub.dev. https://pub.dev/packages/sqflite
2. Flutter Team. (2024). *path package — Flutter & Dart*. pub.dev. https://pub.dev/packages/path
3. Flutter Team. (2024). *Persist data with SQLite — Flutter documentation*. https://docs.flutter.dev/cookbook/persistence/sqlite
4. SQLite Consortium. (2024). *About SQLite*. https://www.sqlite.org/about.html
5. Dart Team. (2024). *Asynchronous programming: futures, async, await — Dart documentation*. https://dart.dev/codelabs/async-await
6. Flutter Team. (2024). *Flutter documentation — State management*. https://docs.flutter.dev/data-and-backend/state-mgmt/intro